

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN – ĐHQG TP.HCM  
KHOA CÔNG NGHỆ THÔNG TIN

# HỆ THỐNG QUẢN TRỊ KINH DOANH

*Xây dựng ứng dụng phân tán theo kiến trúc Microservices*

## Nhóm thực hiện

Vũ Huyền Thiên Lý — 21126028 (35%)

Nguyễn Tuấn Kiệt — 21126027 (25%)

Nguyễn Thế Thanh Long — 22127247 (25%)

Nguyễn Hoàng Danh — 21126057 (15%)

*GVHD: Nguyễn Trường Sơn · Phạm Minh Tú*



# Nội dung trình bày

---

1. Bài toán & Yêu cầu
2. Kiến trúc & Công nghệ
3. Thiết kế dữ liệu
4. Giao tiếp trong hệ phân tán
5. Quy trình phê duyệt (Workflow)
6. Ký điện tử bất đồng bộ
7. Demo giao diện & chức năng
8. Xử lý bài toán phân tán
9. Triển khai (Docker & Kubernetes)
10. Kiểm thử & Truy vết yêu cầu
11. Phân công & Kết luận

01

## Bài toán & Yêu cầu

*Bối cảnh, mục tiêu và phạm vi hệ thống*

# Bối cảnh doanh nghiệp

---

- Công ty ABC: dịch vụ logistics — khai thác cảng, vận chuyển, lưu kho, bốc xếp
- Mỗi tháng phát sinh nhiều hợp đồng, phụ lục, bảng giá, sản lượng, bảng thanh toán
- Nhiều phòng ban tham gia: Kinh doanh, Khai thác, Kế toán, Pháp chế, Giám đốc

## Vấn đề hiện trạng

---

- Xử lý thủ công qua email / Excel / giấy tờ → thông tin phân tán
- Khó kiểm soát trạng thái hồ sơ, khó biết ai đang giữ hồ sơ
- Khó truy vết lịch sử thay đổi; dễ sai lệch khi giá/sản lượng thay đổi
- → Cần một hệ thống tập trung quản lý trọn vòng đời hồ sơ kinh doanh

## Yêu cầu chức năng (mục 4)

---

### Nghịệp vụ lỗi

- Khách hàng & danh mục dịch vụ
- Hợp đồng & phụ lục
- Bảng giá (nhiều phiên bản)
- Sản lượng thực hiện
- Bảng thanh toán

### Xuyên suốt

- Quy trình phê duyệt cấu hình được
- Ký điện tử
- Thông báo bất đồng bộ
- Nhật ký & truy vết
- Phân quyền theo vai trò

## Yêu cầu kỹ thuật (mục 6)

---

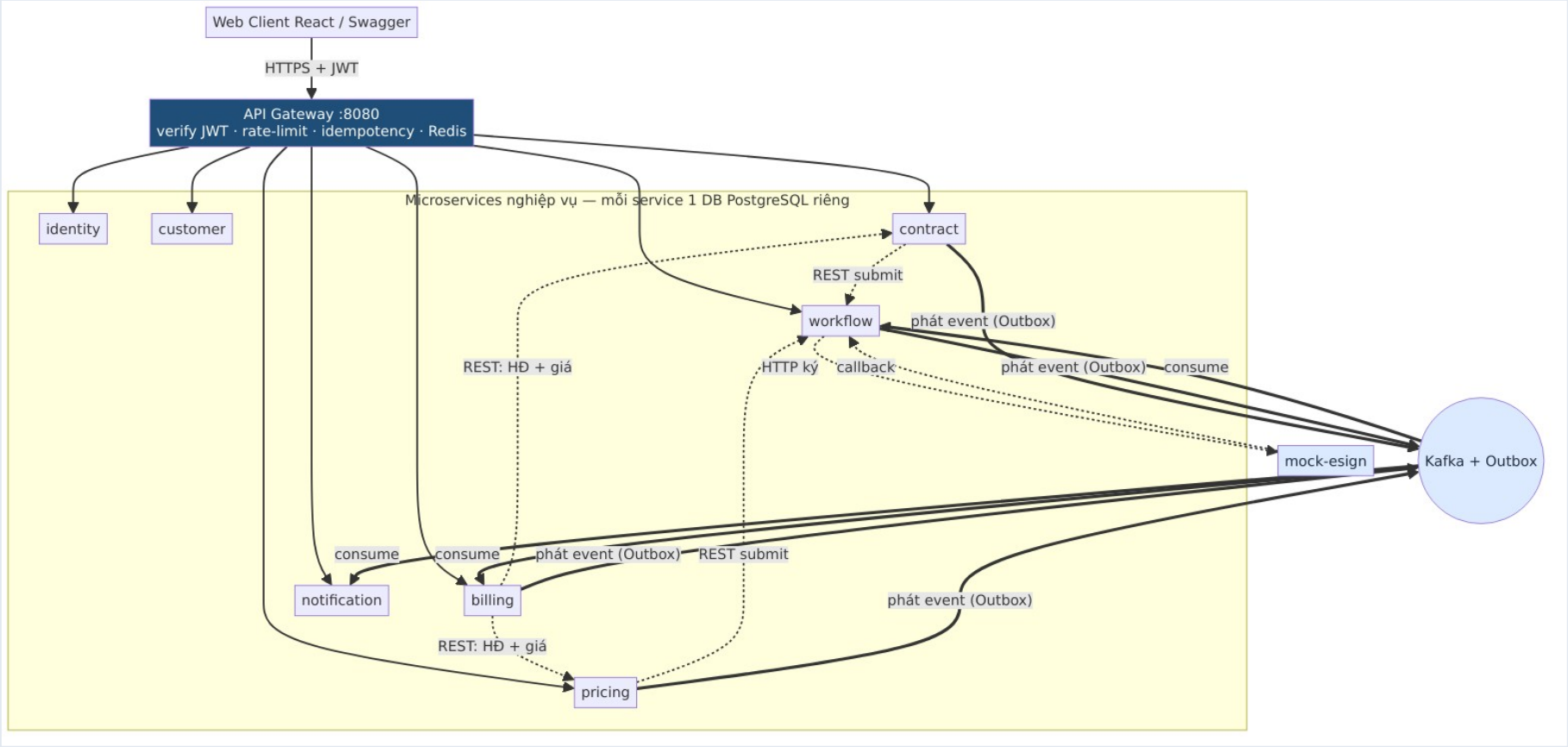
- Kiến trúc Microservices:  $\geq 4$  service nghiệp vụ + API Gateway
- Backend FastAPI, có OpenAPI/Swagger; mỗi service có DB/schema riêng
- PostgreSQL (dữ liệu) · Redis (cache/rate-limit) · Kafka (bất đồng bộ)
- Docker Compose (dev) + Kubernetes (minikube)
- Logging, error handling, validation, phân quyền bằng JWT

02

## Kiến trúc & Công nghệ

*Tổng thể hệ thống và stack sử dụng*

# Kiến trúc tổng thể



Gateway điều phối & xác thực; service gọi REST cho lệnh, phát/nhận sự kiện qua Kafka

# 9 service chạy độc lập

| Service      | Vai trò                                            |
|--------------|----------------------------------------------------|
| api-gateway  | Reverse proxy, verify JWT, rate-limit, idempotency |
| identity     | User, role, JWT (access/refresh), blacklist        |
| customer     | Khách hàng + danh mục dịch vụ                      |
| contract     | Hợp đồng + phụ lục (CTR)                           |
| pricing      | Bảng giá + version (PRC)                           |
| billing      | Sản lượng + bảng thanh toán (PAY)                  |
| workflow     | Engine phê duyệt data-driven + điều phối ký (APR)  |
| notification | Thông báo + nhật ký truy vết                       |
| mock-esign   | Nhà cung cấp ký điện tử giả lập (async)            |

# Công nghệ sử dụng

---

## Backend & dữ liệu

- FastAPI + SQLAlchemy 2 + Pydantic
- PostgreSQL 16 (DB-per-service)
- Redis · Apache Kafka (KRaft)
- JWT + RBAC 7 vai trò

## Frontend & hạ tầng

- React + Vite + TypeScript + Tailwind
- Docker Compose (14 container)
- Kubernetes (minikube)
- OpenAPI/Swagger tự sinh

## Quyết định thiết kế chính

---

- Microservices theo bounded context → mỗi service một trách nhiệm rõ ràng
- Database-per-service → cô lập dữ liệu, không FK xuyên service
- Bất đồng bộ qua Kafka + Outbox → chống mất sự kiện
- Workflow cấu hình bằng dữ liệu → không hard-code quy trình duyệt
- Zero-trust nội bộ: mỗi service vẫn tự verify JWT dù đã qua Gateway

# 03

## Thiết kế dữ liệu

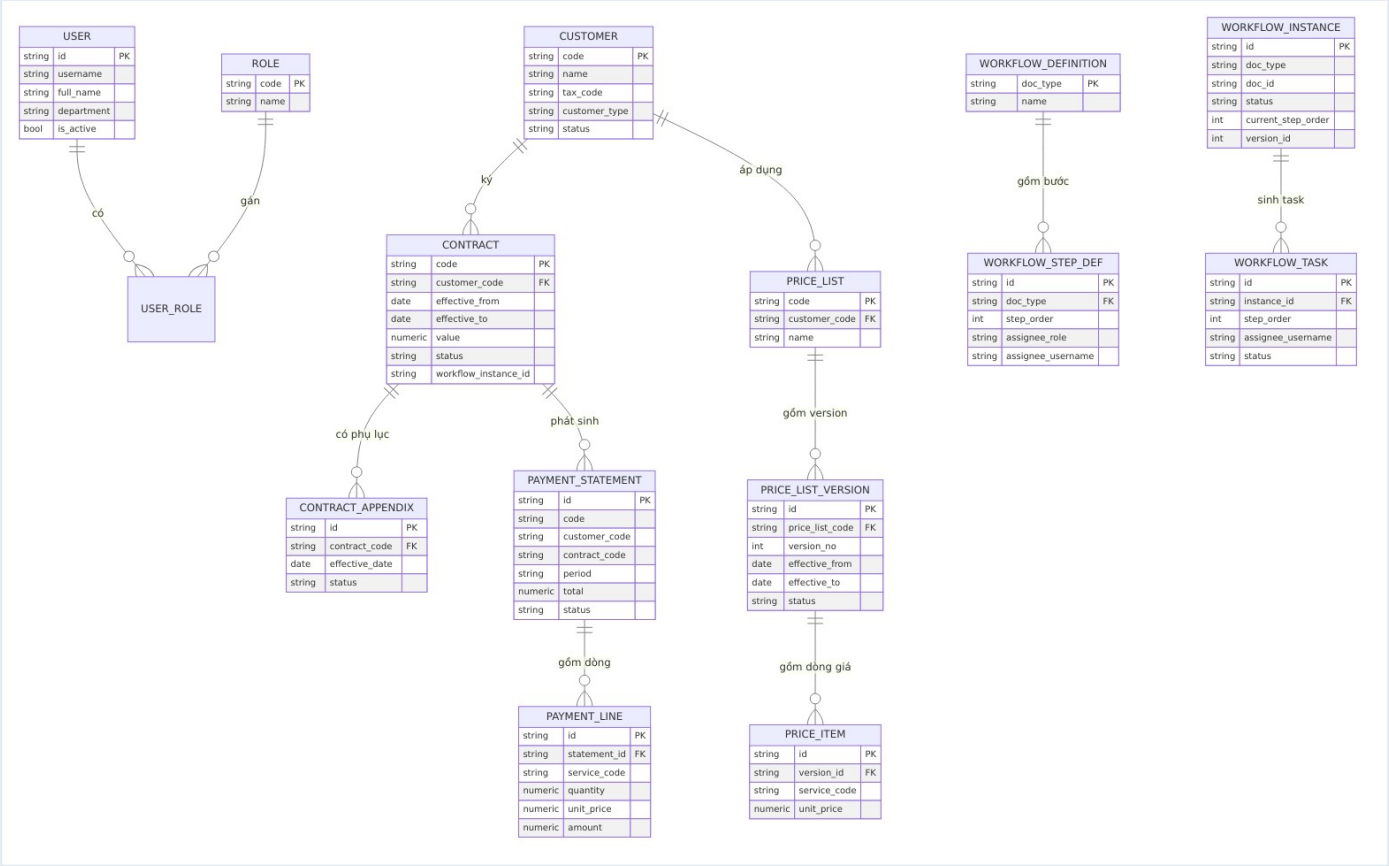
*Mô hình dữ liệu và Database-per-Service*

## Database-per-Service

---

- 1 cụm PostgreSQL, mỗi service 1 database riêng (authdb, contractdb, ...)
- Không FK xuyên service — liên kết bằng mã nghiệp vụ (customer\_code, ...)
- Ưu điểm: cô lập lỗi, độc lập triển khai, dễ mở rộng từng service
- Điểm nhấn (PAY-03): bảng thanh toán COPY cứng đơn giá tại thời điểm tính

# Sơ đồ ERD



Các thực thể chính và quan hệ (theo từng service)

# 04

## Giao tiếp trong hệ phân tán

*Đồng bộ (REST) và bất đồng bộ (Kafka + Outbox)*

## Hai kiểu giao tiếp

---

### Đồng bộ — REST

- Dùng cho lệnh cần kết quả ngay
- Gateway → service; service → service
- VD: submit hồ sơ, billing lấy giá/HĐ

### Bất đồng bộ — Kafka

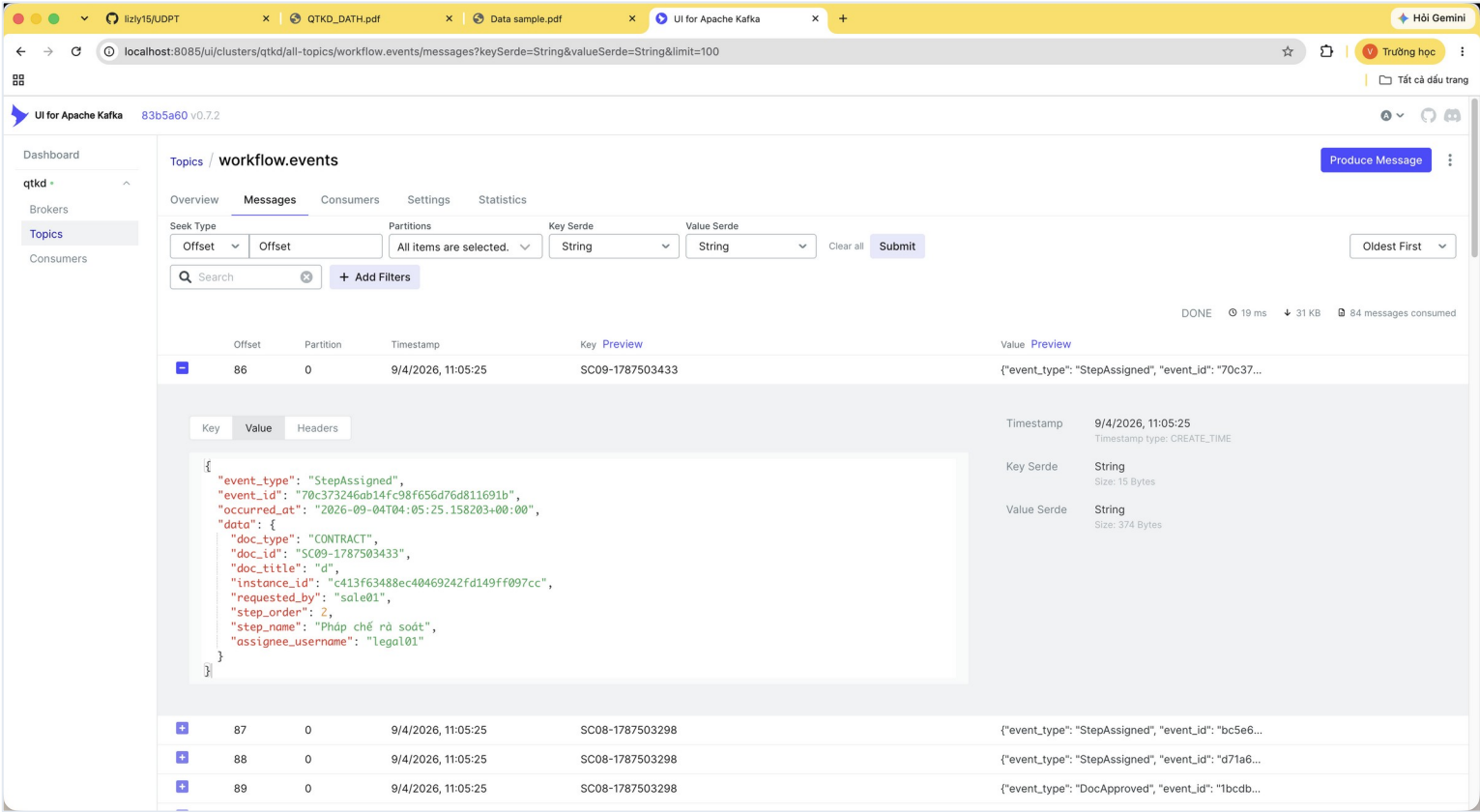
- Dùng cho lan truyền sự kiện
- Thông báo, audit, kết quả duyệt/ký
- Rã kết nối (loose coupling), chịu lỗi tốt

## Outbox Pattern — chống mất sự kiện

---

- Vấn đề: ghi DB thành công nhưng phát Kafka lỗi → mất sự kiện
- Giải pháp: ghi thay đổi + bản ghi outbox trong CÙNG một transaction
- Một relay đọc outbox → phát Kafka → đánh dấu đã gửi
- Consumer idempotent → an toàn khi phát lại

# Sự kiện thật trên Kafka



- 5 topic: contract / pricing / billing / workflow / esign
- Mỗi message có event\_id, occurred\_at
- Phát qua Outbox, consumer xử lý idempotent

05

## Quy trình phê duyệt

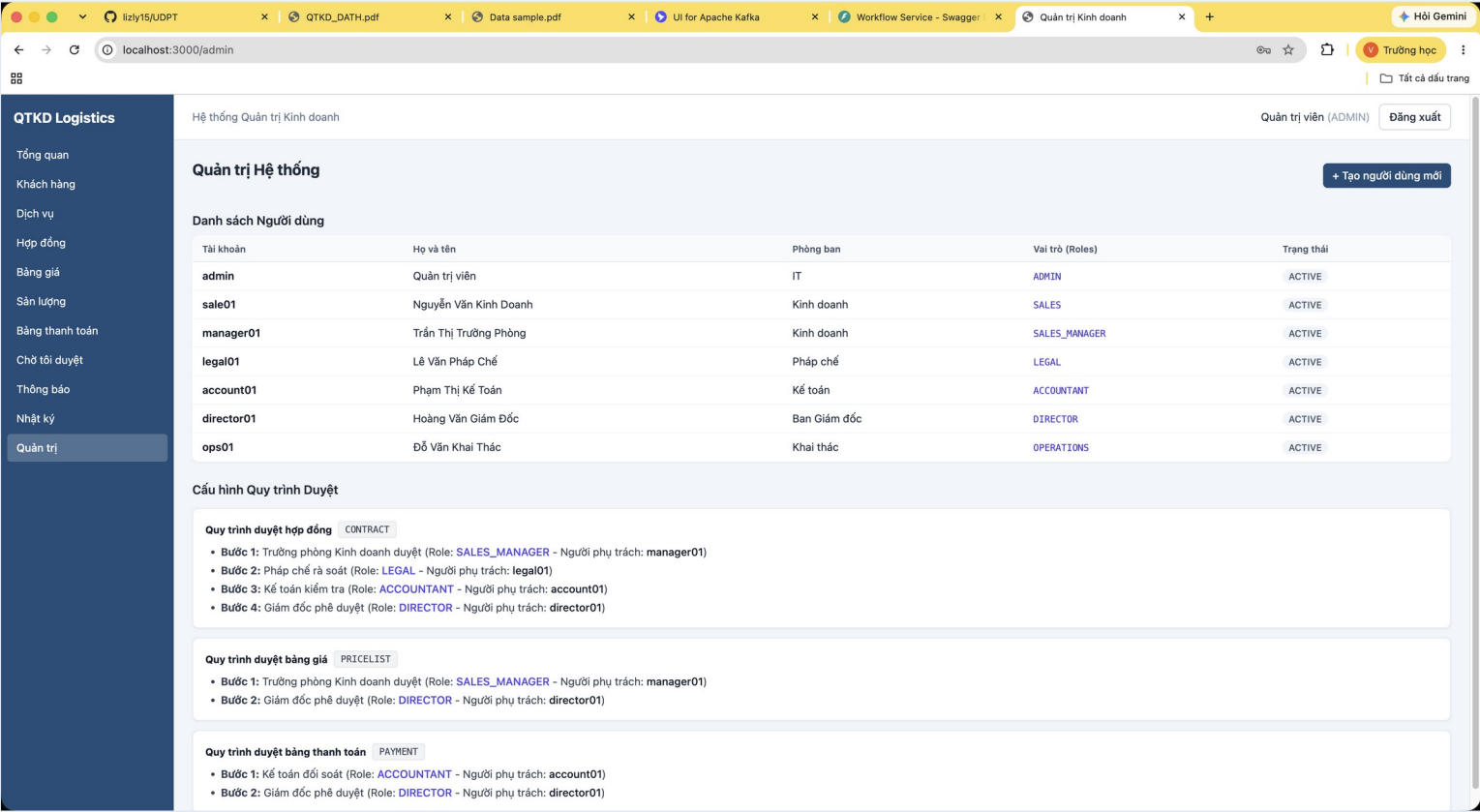
*Workflow engine cấu hình bằng dữ liệu*

## Workflow data-driven (không hard-code)

---

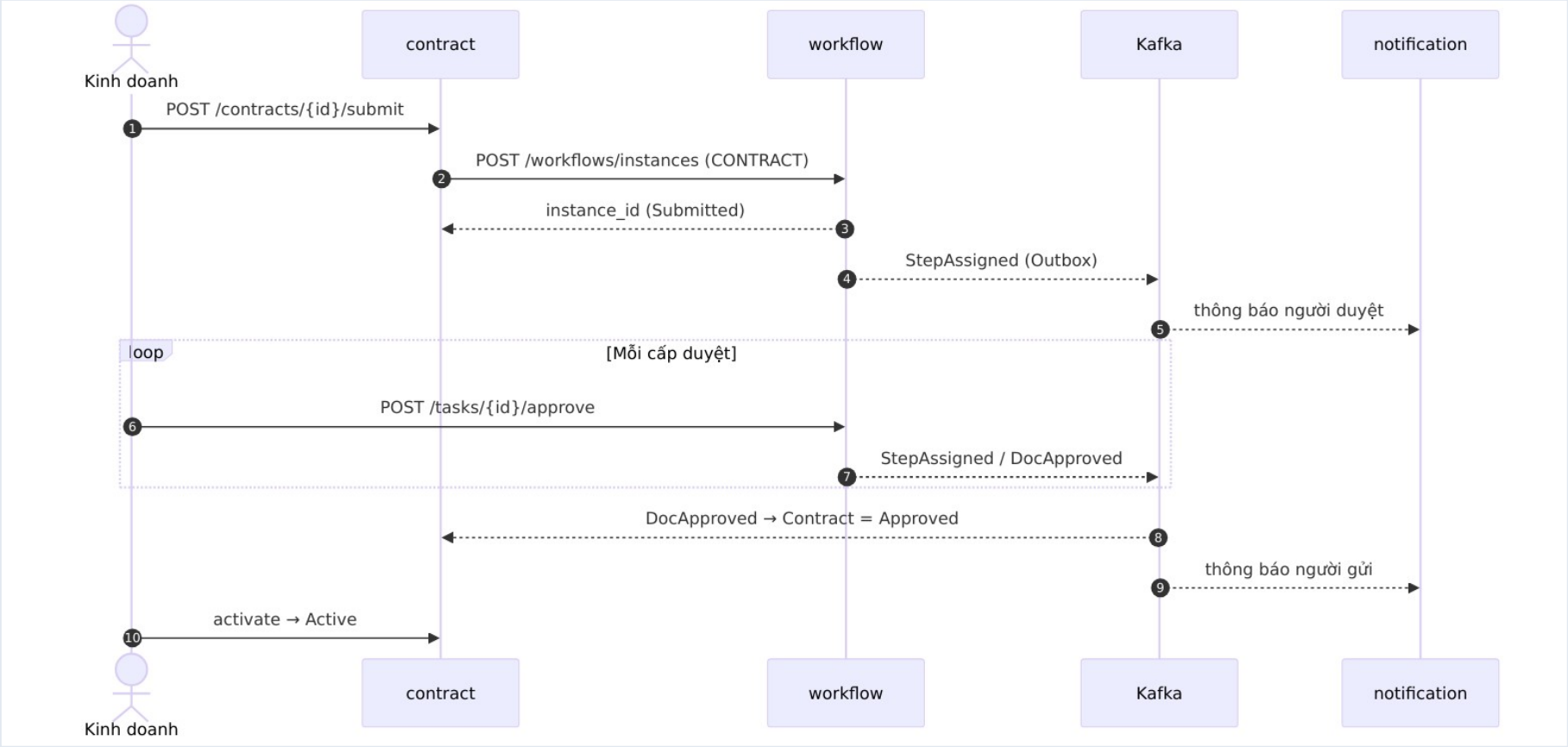
- Quy trình lưu trong bảng: định nghĩa theo loại hồ sơ + các bước + người duyệt
- Engine đọc DB để quyết định bước kế tiếp — KHÔNG viết if/else theo loại hồ sơ
- Mỗi loại hồ sơ (Hợp đồng / Bảng giá / Thanh toán) có quy trình riêng
- Thêm/sửa quy trình = sửa dữ liệu, không cần sửa code

# Cấu hình quy trình duyệt



- Quản trị người dùng + vai trò
- Xem cấu hình từng quy trình
- Hợp đồng: 4 cấp; Thanh toán: 2 cấp

# Trình tự duyệt hợp đồng



Submit → duyệt các cấp → phát DocApproved (Kafka) → hồ sơ chuyển Approved

## Các quy tắc phê duyệt (APR)

---

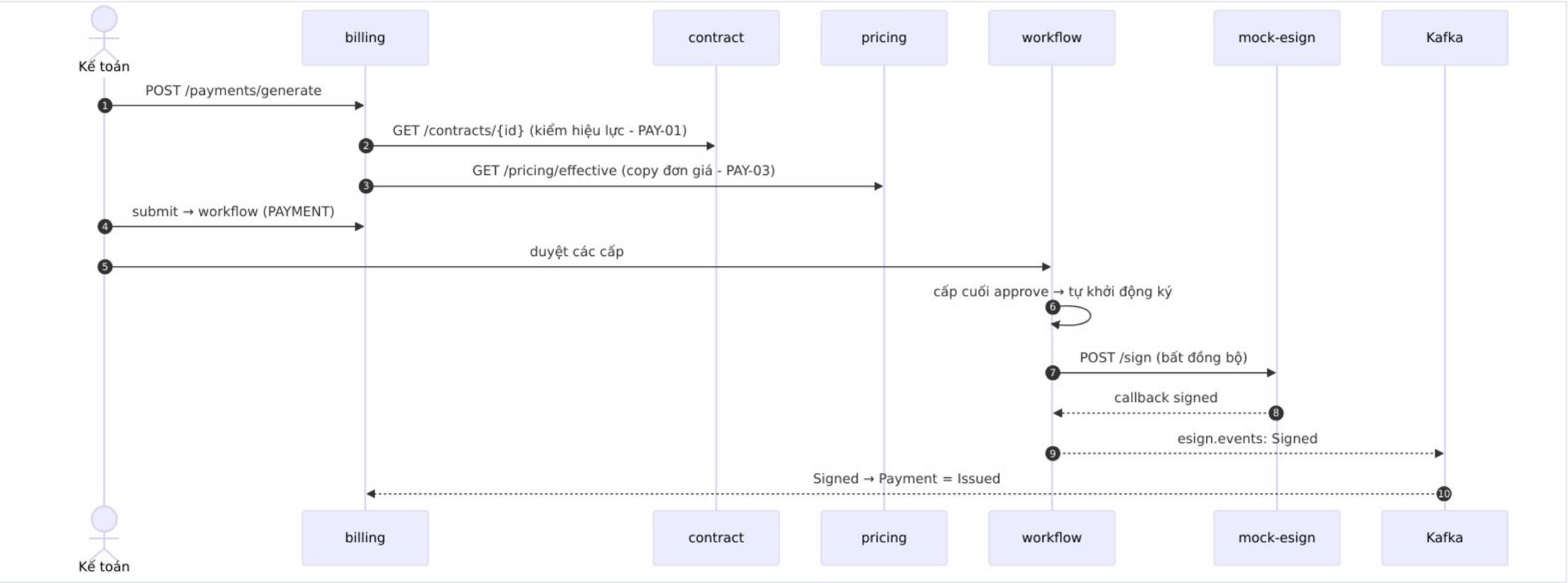
- APR-01: chỉ đúng người được giao ở bước hiện tại (không chỉ kiểm role)
- APR-02: không duyệt nhảy bước / duyệt lại bước đã xong
- APR-03: reject / yêu cầu sửa bắt buộc nhập lý do
- APR-05: bước cuối duyệt → phát sự kiện để xử lý tiếp (vd gửi ký)

06

## Ký điện tử

*Tích hợp bất đồng bộ với callback*

# Luồng ký điện tử bất đồng bộ



*Bảng thanh toán duyệt xong → workflow tự gửi ký → mock-esign callback → Issued*

## Quản lý phiên ký & xử lý lỗi

---

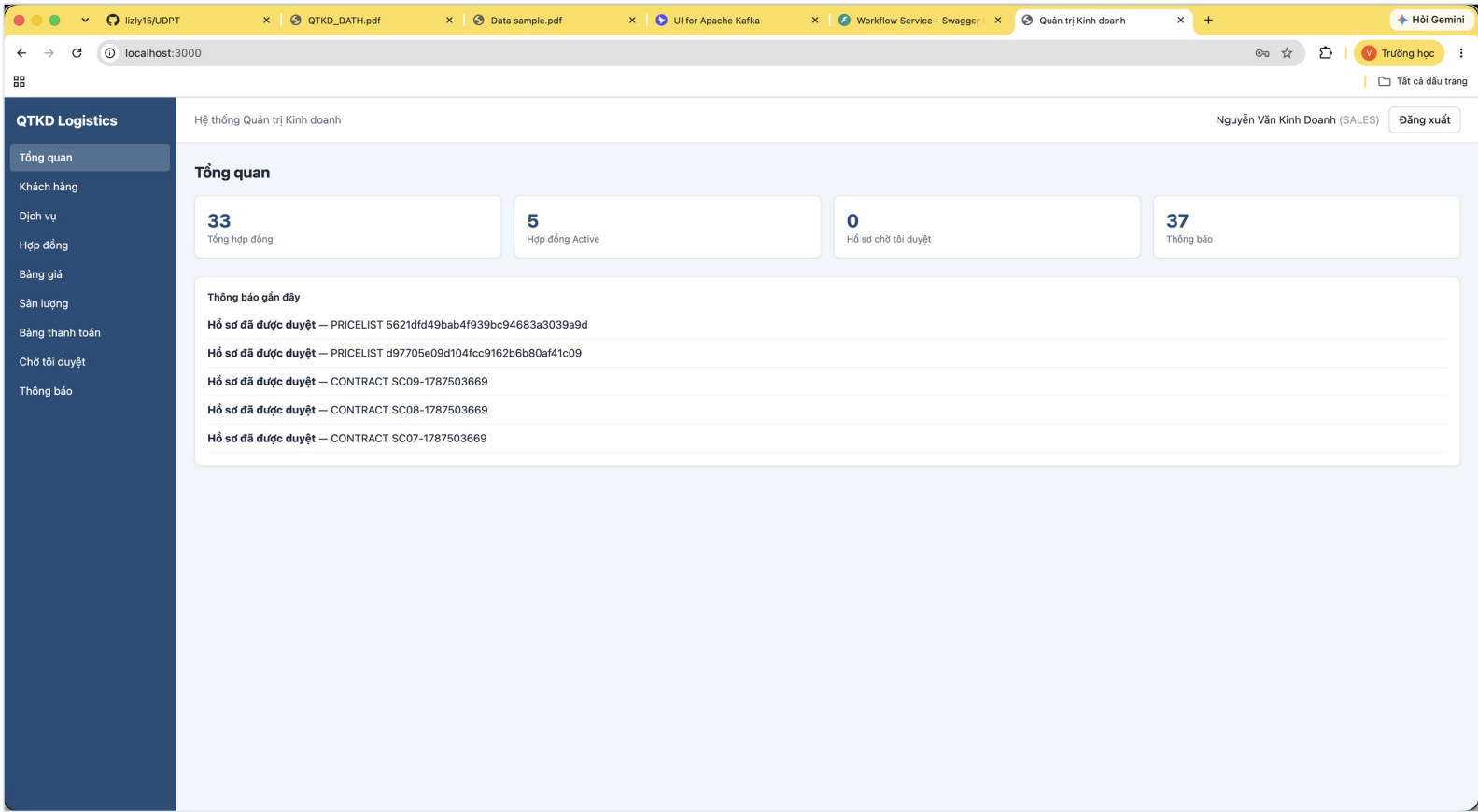
- Trạng thái phiên ký TÁCH BIỆT trạng thái duyệt (pending/signing/signed/failed)
- Chỉ gửi ký sau khi hồ sơ được duyệt nội bộ (PAY-06)
- Ký thất bại → phản ánh rõ + cho phép GỬI KÝ LẠI (PAY-07, SC-06)
- Nếu dịch vụ ký lỗi tạm thời → có retry, không hỏng dữ liệu đã duyệt

07

## Demo giao diện & chức năng

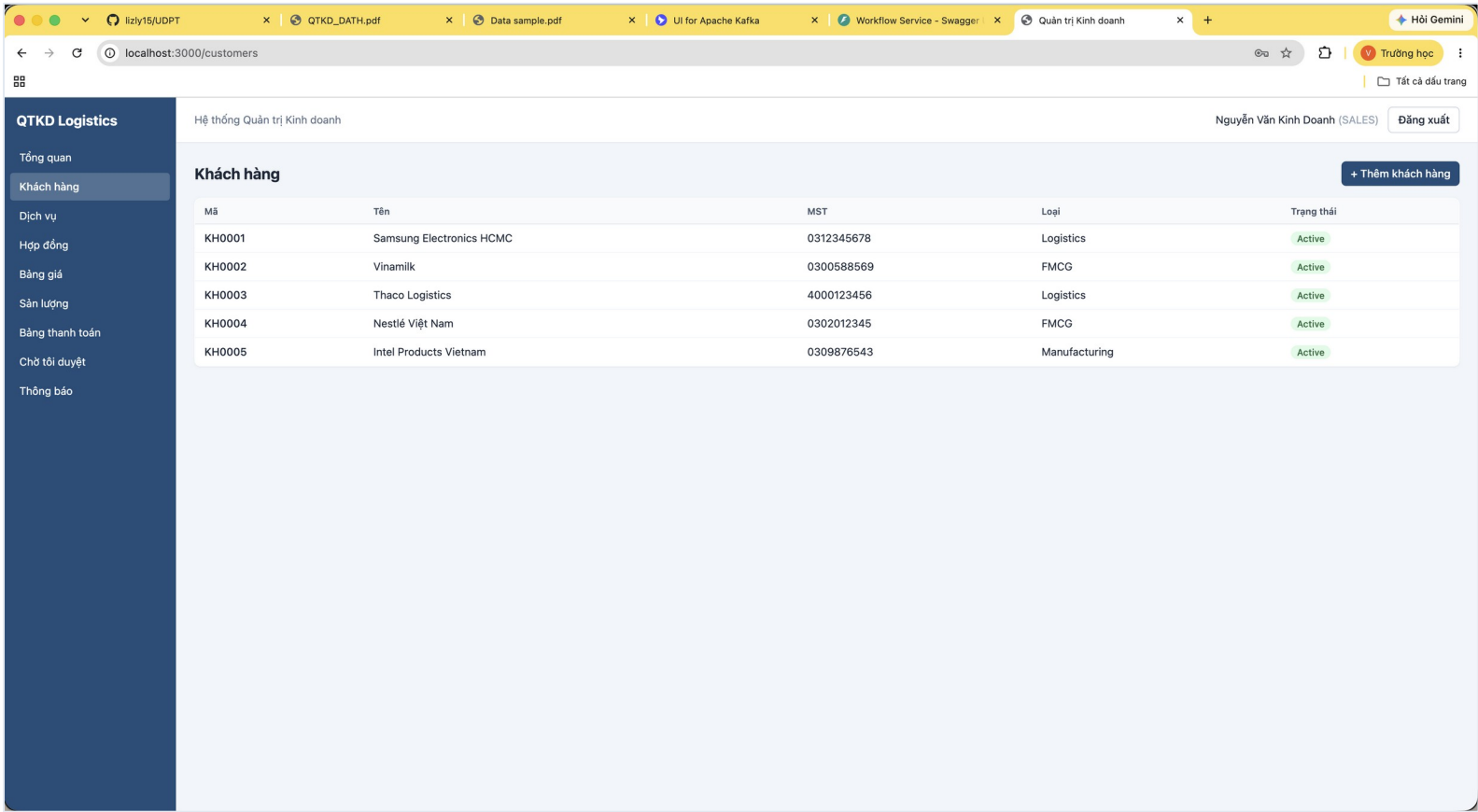
*Minh chứng các chức năng nghiệp vụ*

# Đăng nhập & Tổng quan



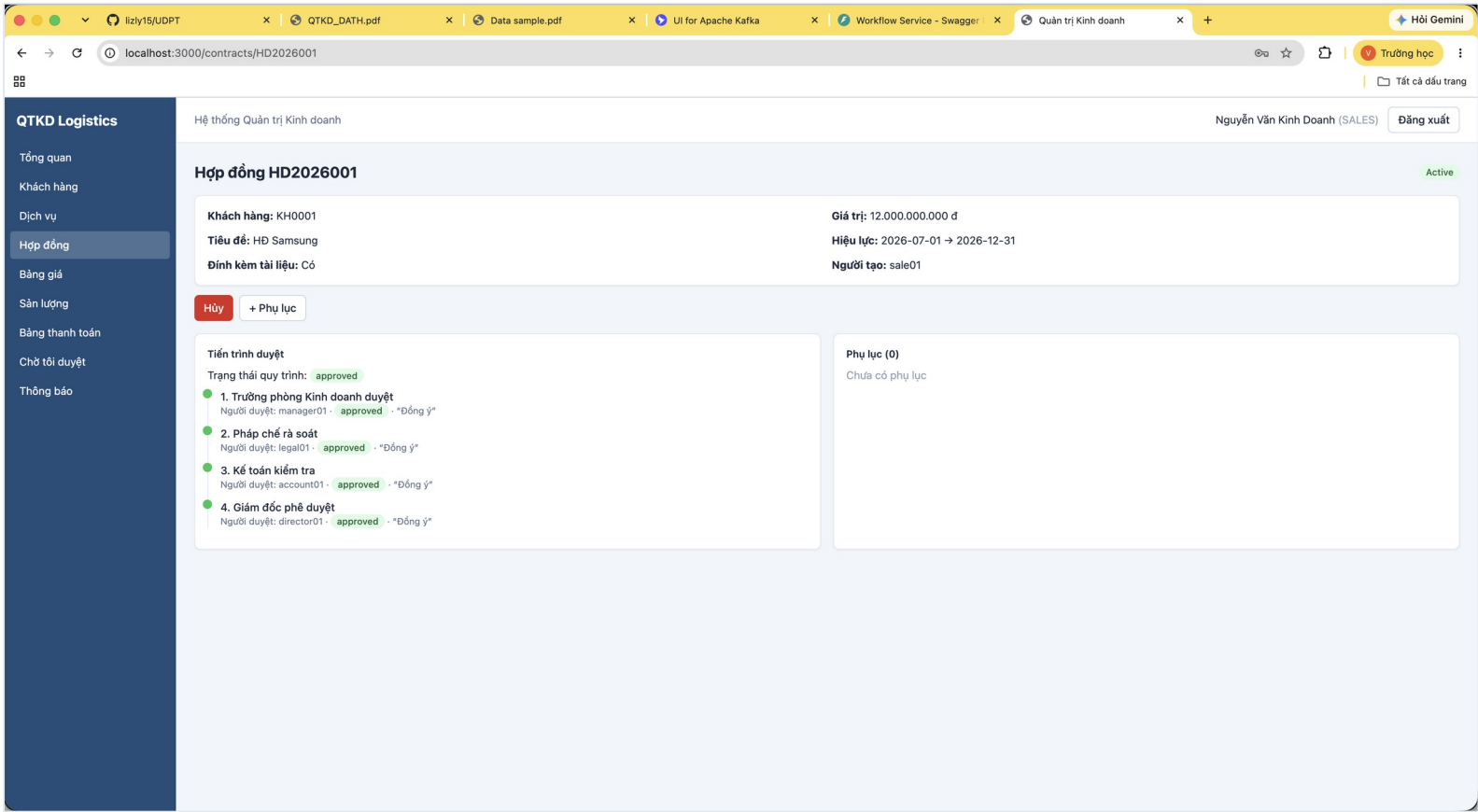
Xác thực JWT; dashboard tổng hợp số liệu theo vai trò

# Quản lý khách hàng



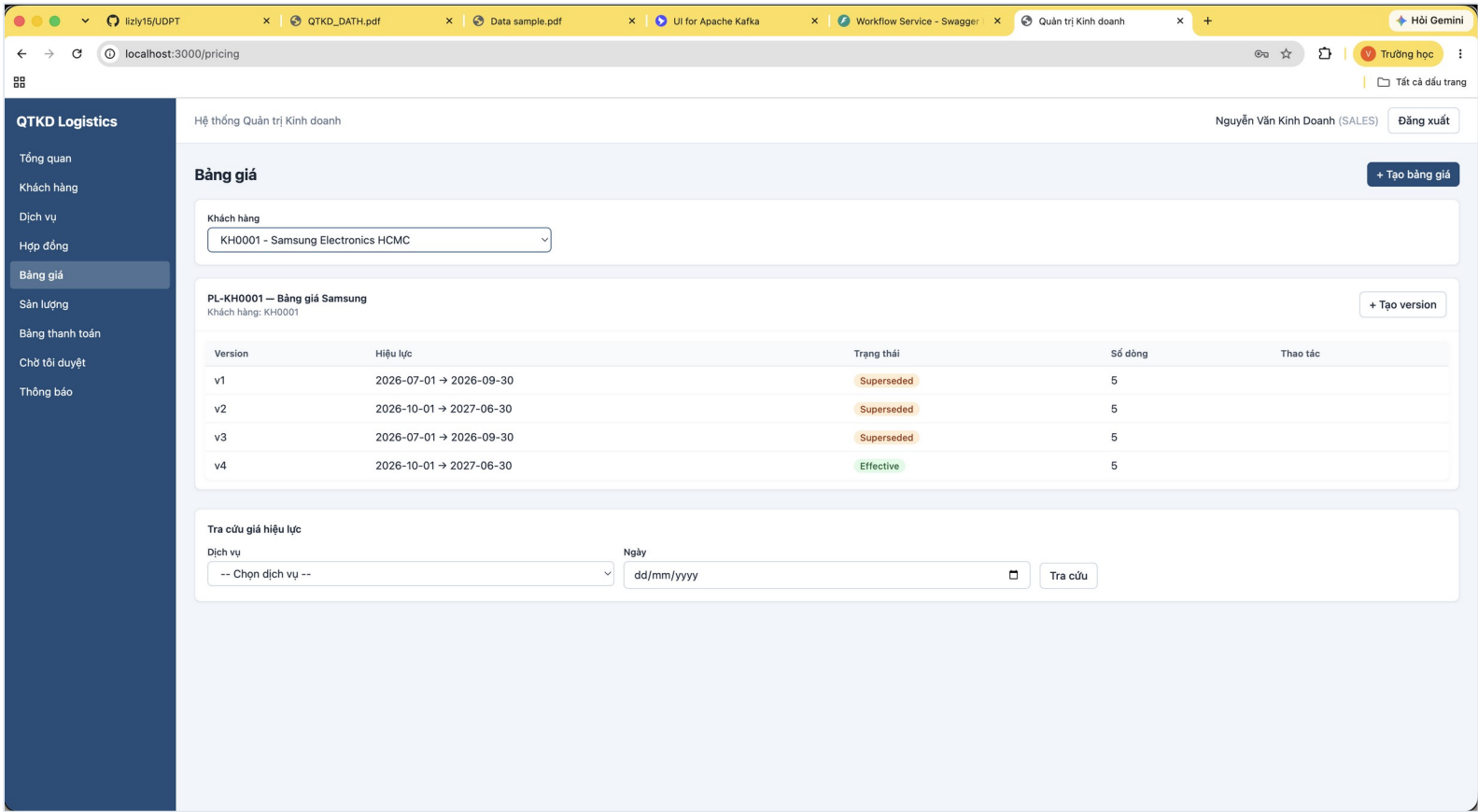
Chức năng 4.1 — CRUD khách hàng, danh mục dịch vụ

# Hợp đồng & tiến trình duyệt



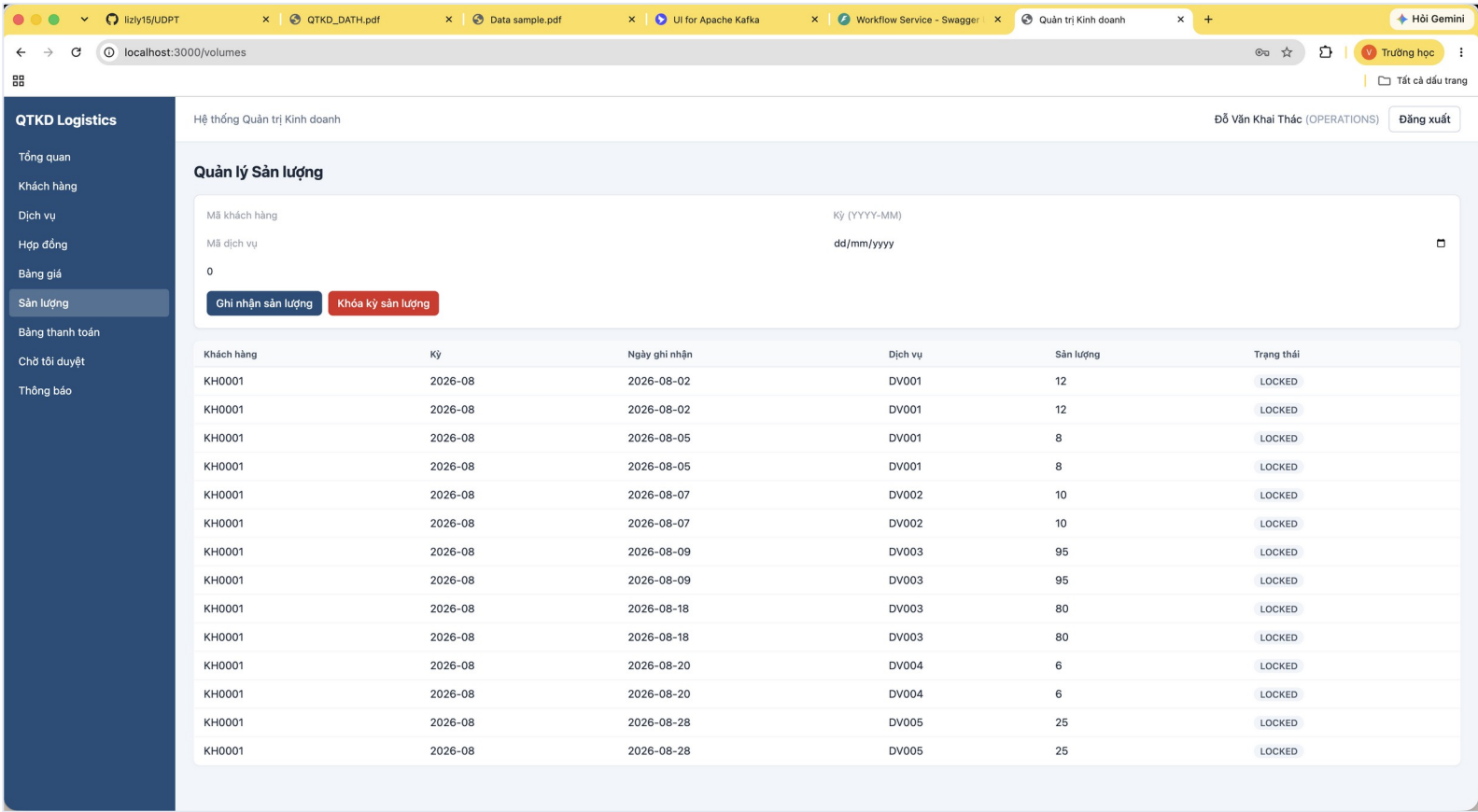
Chức năng 4.2/4.3 — chi tiết hợp đồng + tiến trình duyệt 4 cấp

# Bảng giá nhiều phiên bản



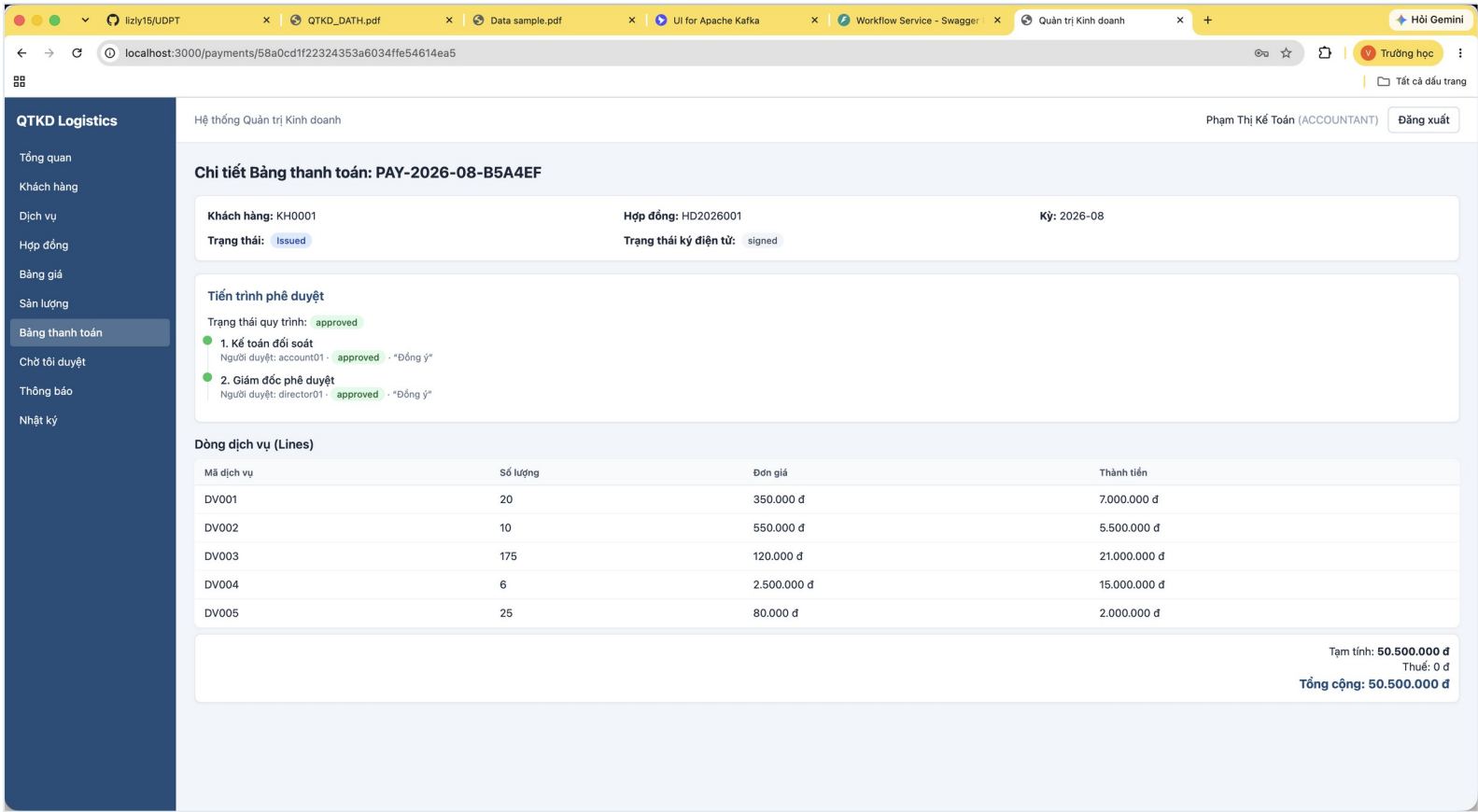
Chức năng 4.4 — version Effective/Superseded, tra giá theo ngày

# Sản lượng & khóa kỳ



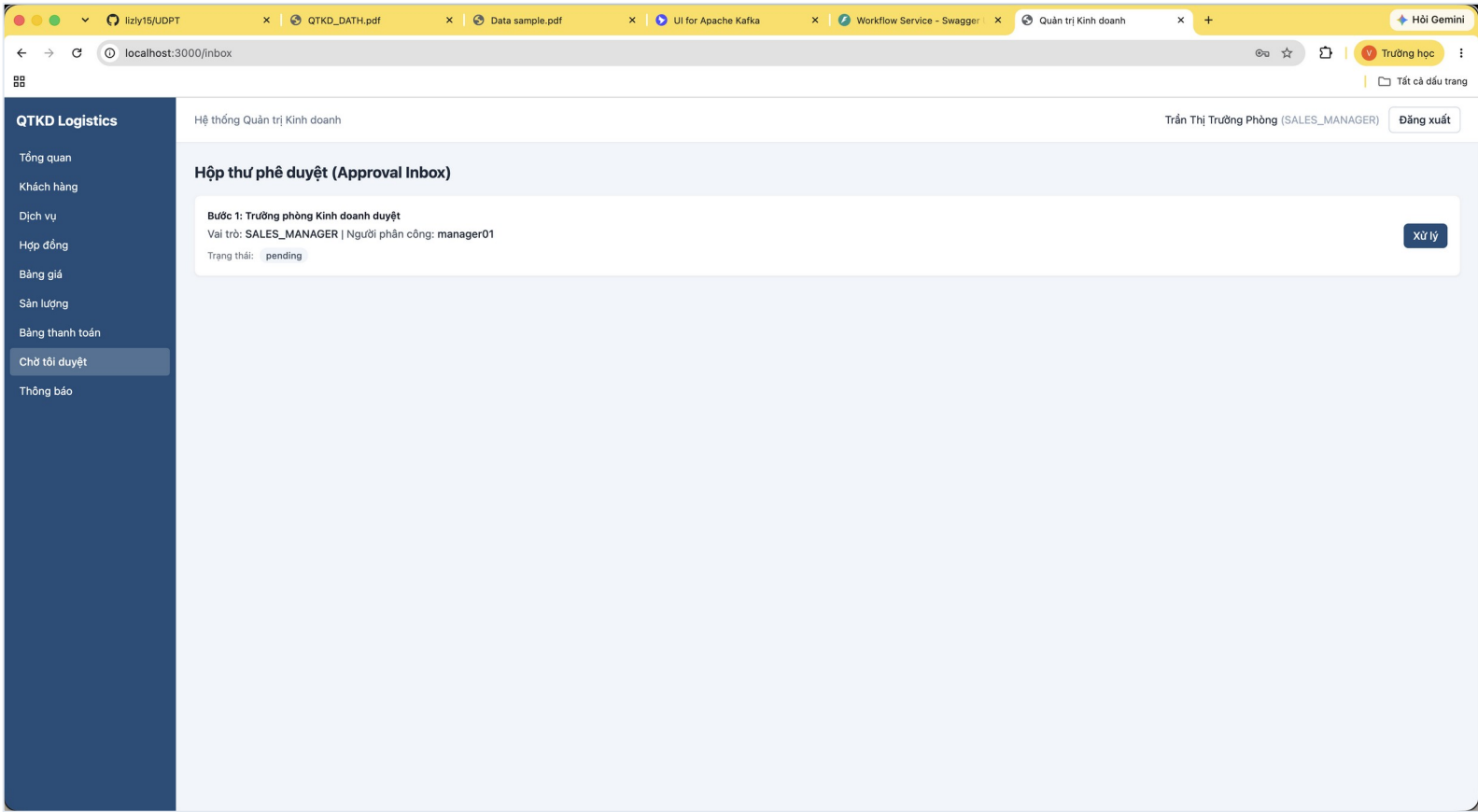
Chức năng 4.5 — ghi nhận sản lượng, khóa kỳ trước khi tính phí

# Bảng thanh toán (khớp Phụ lục A.8)



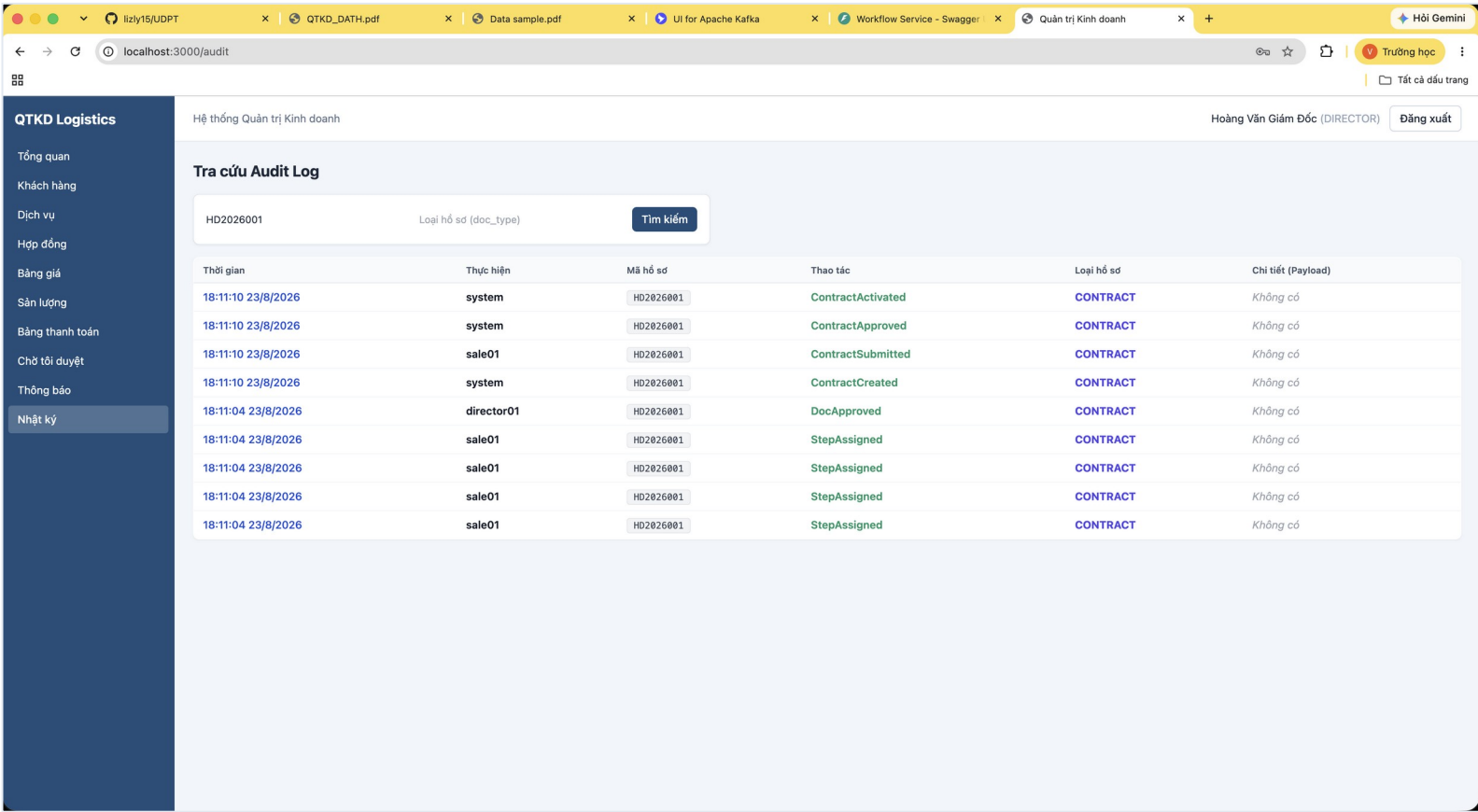
Chức năng 4.6/4.8 — tổng 50.500.000đ, trạng thái Issued, ký điện tử signed

# Hộp thư phê duyệt



Chức năng 4.7 — chỉ đúng người được giao mới thấy & xử lý tác vụ

# Thông báo & Nhật ký



Chức năng 4.9/4.10 — thông báo bất đồng bộ + truy vết đầy đủ theo hồ sơ

08

# Xử lý bài toán phân tán

*Các điểm nhấn kỹ thuật*

## 7 vấn đề phân tán & giải pháp

| Vấn đề                           | Giải pháp                                   |
|----------------------------------|---------------------------------------------|
| Quy trình không hard-code        | Engine đọc định nghĩa từ DB                 |
| Double submit (SC-09)            | Idempotency-Key (Redis) + instance duy nhất |
| Race condition duyệt (SC-05)     | Optimistic locking (version) → 409          |
| Mất sự kiện                      | Outbox Pattern + consumer idempotent        |
| Phân quyền theo ngữ cảnh (SC-08) | Kiểm tra đúng assignee, không chỉ role      |
| Dữ liệu lịch sử (SC-04/10)       | Chọn giá theo ngày + copy đơn giá cứng      |
| Service phụ lỗi (SC-07)          | Async + Outbox, nghiệp vụ chính không hỏng  |

# Chặn sai quy tắc nghiệp vụ

The screenshot shows a web browser window with multiple tabs. The active tab is 'Quản trị Kinh doanh' (Business Management). The URL is 'localhost:3000/contracts/HD-DEMO'. The page title is 'Hợp đồng HD-DEMO'. The user is logged in as 'Nguyễn Văn Kinh Doanh (SALES)' with a 'Đăng xuất' (Logout) button. The left sidebar contains a menu with items: 'Tổng quan', 'Khách hàng', 'Dịch vụ', 'Hợp đồng', 'Bảng giá', 'Sản lượng', 'Bảng thanh toán', 'Chờ tài duyệt', and 'Thông báo'. The main content area shows a contract form with the following fields and values:

| Field             | Value                   |
|-------------------|-------------------------|
| Khách hàng        | KH0001                  |
| Tiêu đề           | HD demo                 |
| Đính kèm tài liệu | Chưa                    |
| Giá trị           | 1.000.000.000 đ         |
| Hiệu lực          | 2026-07-01 → 2026-12-31 |
| Người tạo         | sale01                  |

Below the form, there are two sections:

- Tiến trình duyệt**: HỒ SƠ CHƯA ĐƯỢC GỬI DUYỆT.
- Phụ lục (0)**: Chưa có phụ lục.

At the bottom of the form, there are three buttons: 'Sửa', 'Gửi duyệt', and 'Hủy'.

- CTR-02/SC-01: thiếu tài liệu → chặn submit
- Rule enforce ở tầng service
- Trả mã lỗi rõ ràng, chuẩn hóa

09

# Triển khai

*Docker Compose và Kubernetes*

# Docker Compose (môi trường dev)

```
→ DATH git:(main) docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED    STATUS    PORTS
qtkd-api-gateway-1  qtkd-api-gateway    "sh -c 'uvicorn app..." api-gateway  11 minutes ago Up 10 minutes 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
qtkd-billing-service-1  qtkd-billing-service    "sh -c 'uvicorn app..." billing-service  11 minutes ago Up 10 minutes 0.0.0.0:8005->8005/tcp, [::]:8005->8005/tcp
qtkd-contract-service-1  qtkd-contract-service    "sh -c 'uvicorn app..." contract-service  11 minutes ago Up 10 minutes 0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp
qtkd-customer-service-1  qtkd-customer-service    "sh -c 'uvicorn app..." customer-service  11 minutes ago Up 10 minutes 0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp
qtkd-frontend-1        qtkd-frontend        "/docker-entrypoint..." frontend  11 minutes ago Up 10 minutes 0.0.0.0:3000->80/tcp, [::]:3000->80/tcp
qtkd-identity-service-1  qtkd-identity-service    "sh -c 'uvicorn app..." identity-service  11 minutes ago Up 10 minutes 0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
qtkd-kafka-1          apache/kafka:3.7.0    "/_cacert_entrypoint..." kafka  11 days ago Up 11 minutes 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
qtkd-kafka-ui-1        provectuslabs/kafka-ui:latest  "/bin/sh -c 'java --..." kafka-ui  11 days ago Up 11 minutes 0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
qtkd-mock-esign-1      qtkd-mock-esign        "sh -c 'uvicorn app..." mock-esign  11 minutes ago Up 10 minutes 0.0.0.0:8009->8009/tcp, [::]:8009->8009/tcp
qtkd-notification-service-1  qtkd-notification-service    "sh -c 'uvicorn app..." notification-service  11 minutes ago Up 10 minutes 0.0.0.0:8007->8007/tcp, [::]:8007->8007/tcp
qtkd-postgres-1        postgres:16-alpine    "docker-entrypoint.s..." postgres  11 days ago Up 11 minutes (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
qtkd-pricing-service-1  qtkd-pricing-service    "sh -c 'uvicorn app..." pricing-service  11 minutes ago Up 10 minutes 0.0.0.0:8004->8004/tcp, [::]:8004->8004/tcp
qtkd-redis-1          redis:7-alpine        "docker-entrypoint.s..." redis  11 days ago Up 11 minutes 0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
qtkd-workflow-service-1  qtkd-workflow-service    "sh -c 'uvicorn app..." workflow-service  11 minutes ago Up 10 minutes 0.0.0.0:8006->8006/tcp, [::]:8006->8006/tcp
→ DATH git:(main)
```

- 14 container chạy độc lập
- 1 lệnh: make up
- Postgres · Redis · Kafka · 9 service

# Kubernetes (minikube)

```

• → DATH git:(main) ✗ kubectl -n qtkd wait --for=condition=available --timeout=300s deployment --all && kubectl get pods -n qtkd
deployment.apps/api-gateway condition met
deployment.apps/billing-service condition met
deployment.apps/contract-service condition met
deployment.apps/customer-service condition met
deployment.apps/identity-service condition met
deployment.apps/kafka condition met
deployment.apps/mock-esign condition met
deployment.apps/notification-service condition met
deployment.apps/postgres condition met
deployment.apps/pricing-service condition met
deployment.apps/redis condition met
deployment.apps/workflow-service condition met
NAME                                READY   STATUS    RESTARTS   AGE
api-gateway-6c766cfb5b-zdktm        1/1     Running   0           103s
billing-service-5d9fdddc-99mms       1/1     Running   0           103s
contract-service-85f6b56978-hmqd7   1/1     Running   0           103s
customer-service-cbf55498c-95h76    1/1     Running   0           103s
identity-service-8dd5d7cb7-7bxt6    1/1     Running   0           103s
kafka-848c88c8cd-blqx6              1/1     Running   4 (58s ago) 6m10s
mock-esign-6d75d9d9b7-fkcr7         1/1     Running   0           103s
notification-service-84cd56c79-26fk7 1/1     Running   0           103s
postgres-6589667d9f-5pzqw           1/1     Running   0           6m10s
pricing-service-567c54df9b-rhtwn     1/1     Running   0           103s
redis-5d85cc97f8-scfnx              1/1     Running   0           6m10s
workflow-service-6994659cfb-7xg25    1/1     Running   0           103s
○ → DATH git:(main) ✗

```

- Manifests: namespace, config, hạ tầng, 9 service
- Toàn bộ pod Running
- Deploy được trên minikube

10

## Kiểm thử & Truy vết

*Chất lượng và bao phủ yêu cầu*

## Kiểm thử kịch bản (SC-01 → SC-10)

```
→ DATH git:(main) ✕ make smoke
bash scripts/smoke-test.sh
===== SMOKE TEST SC-01..SC-10 =====
SC-01: Tạo hợp đồng chưa đính kèm -> không cho Submit
  ✓ SC-01 chặn submit thiếu đính kèm
SC-02: Hai bảng giá chống thời gian hiệu lực -> báo lỗi
  ✓ SC-02 chặn chống hiệu lực
SC-03: Lập bảng thanh toán khi hợp đồng hết hạn -> không cho phép
  ✓ SC-03 chặn thanh toán HĐ hết hạn
SC-04: Đổi bảng giá sau phát hành -> bảng thanh toán cũ giữ nguyên đơn giá
  ✓ SC-04 đơn giá DV003 đã copy giữ nguyên 120000
SC-05: Hai người cùng Approve -> chỉ một thành công
  ✓ SC-05 duyệt lại bước đã xong bị chặn
SC-06: Ký điện tử -> có endpoint gửi ký lại
  ✓ SC-06 endpoint retry ký hoạt động
SC-07: Notification Service lỗi -> nghiệp vụ chính KHÔNG lỗi + bù sự kiện khi phục hồi
  ✓ SC-07 tạo hợp đồng khi notif chết vẫn OK
  ✓ SC-07 submit khi notif chết vẫn OK
  ✓ SC-07 notif phục hồi & bù được audit (3 bản ghi)
SC-08: Sai assignee Approve -> từ chối
  ✓ SC-08 sai assignee bị chặn
SC-09: Submit nhiều lần -> chỉ một workflow
  ✓ SC-09 submit lần 1 OK
  ✓ SC-09 submit lần 2 bị chặn (409)
SC-10: Phụ lục hiệu lực 01/10 nhưng tính tháng 09 -> dùng giá cũ
  ✓ SC-10 tháng 09 dùng giá cũ 120000
  ✓ SC-10 tháng 10 dùng giá mới 150000
===== KẾT QUẢ: 14 PASS / 0 FAIL =====
→ DATH git:(main) ✕
```

- Smoke test end-to-end
- 14/14 assertion PASS
- Bao phủ toàn bộ Phụ lục A.12

# Unit test (pytest)

```

• → DATH git:(main) ✕ make test
docker run --rm -v "/Users/hahoang/Tly/UDPT/DATH/services":/app -w /app python:3.12-slim sh -c \
'pip install -q -r requirements-test.txt && \
  for s in contract-service pricing-service workflow-service billing-service; do \
    echo "== $s =="; python -m pytest $s/tests -q -p no:cacheprovider || exit 1; done'
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering
your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if y
ou know what you are doing and want to suppress this warning.

[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
== contract-service ==
..... [100%]
12 passed in 0.12s
== pricing-service ==
..... [100%]
6 passed in 0.12s
== workflow-service ==
..... [100%]
7 passed in 0.14s
== billing-service ==
..... [100%]
6 passed in 0.12s
○ → DATH git:(main) ✕

```

- State machine & business rules
- 31/31 test PASS
- 4 service: contract, pricing, workflow, billing

## Bao phủ yêu cầu (truy vết)

---

- Chức năng 4.1–4.10: đầy đủ, có minh chứng giao diện
- Business rules CTR / PRC / PAY / APR: enforce ở tầng service
- Kịch bản SC-01 → SC-10: 14/14 PASS (tự động)
- Yêu cầu kỹ thuật mục 6: Microservices, Gateway, PostgreSQL, Redis, Kafka, Docker, Kubernetes, JWT — đủ

11

# Phân công & Kết luận

*Đóng góp và hướng phát triển*

# Phân công công việc

| Thành viên            | %   | Công việc                                                                                                 |
|-----------------------|-----|-----------------------------------------------------------------------------------------------------------|
| Vũ Huyền Thiên Lý     | 35% | Kiến trúc; Gateway, Identity, Contract, Pricing, Workflow, Notification; hạ tầng, kiểm thử, báo cáo/slide |
| Nguyễn Tuấn Kiệt      | 25% | Frontend: khung + Khách hàng, Dịch vụ, Hợp đồng, Bảng giá                                                 |
| Nguyễn Thế Thanh Long | 25% | Frontend: Sản lượng, Thanh toán, Phê duyệt, Thông báo, Nhật ký, Quản trị                                  |
| Nguyễn Hoàng Danh     | 15% | Backend: Customer, Billing, Mock e-sign                                                                   |

# Kết luận

---

- Hiện thực đầy đủ nghiệp vụ quản trị kinh doanh trên kiến trúc microservices
- Giải quyết trọn các bài toán phân tán: Outbox, idempotency, optimistic lock, phân quyền ngữ cảnh, ký điện tử async
- Triển khai thành công trên Docker Compose và Kubernetes
- Kiểm chứng bằng bộ test tự động: 14/14 smoke + 31/31 unit

## Hướng phát triển

---

- Thay Outbox relay bằng CDC (Debezium)
- Thêm distributed tracing (OpenTelemetry) để quan sát xuyên service
- Auto-scaling (HPA) và lưu trữ bền (PVC/StatefulSet) trên Kubernetes
- Tích hợp dịch vụ ký điện tử thật thay cho mock

# CẢM ƠN THẦY VÀ CÁC BẠN ĐÃ LẮNG NGHE

Mã nguồn: [github.com/lizly15/UDPT](https://github.com/lizly15/UDPT)

Q&A